

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

**Факультет Информационных технологий
Кафедра Информатики и информационных технологий**

**направление подготовки
09.03.02 «Информационные системы и технологии»**

ПРАКТИЧЕСКАЯ РАБОТА № 11

Дисциплина: «BackEnd-разработка»

Тема: Работа с базами данных в приложении на основе ASP.NET Core

Выполнил(а): студент(ка) группы 241-333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись _____
(Дата) (Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

**Москва
2026**

Цель работы:

Ознакомиться с использованием ORM Entity Framework в приложениях на платформе ASP.NET Core.

Задание:

1. Создать проект ASP.NET Core Web API или использовать ранее созданный проект из предыдущей лабораторной работы.
2. Спроектировать базу данных, состоящую минимум из трёх сущностей, связанных между собой.
3. Написать слой Data Access Layer, используя Entity Framework Core (либо подход Code First, либо подход Database First).
4. Реализовать (или использовать уже существующие) API-методы для работы с данными. Минимум: реализовать полный CRUD.
5. Протестировать работу API через Swagger или Postman.

1. Написание приложения

1. Для создания проекта использовался шаблон **ASP.NET Core Web API** в среде Visual Studio 2026 с целевой платформой **.NET 10**. В проект подключены пакеты `Microsoft.EntityFrameworkCore`, `Pomelo.EntityFrameworkCore.MySql`, `Microsoft.EntityFrameworkCore.Tools` и `Microsoft.EntityFrameworkCore.Design`.
2. **Проектирование базы данных.** В качестве предметной области выбран интернет-магазин. Спроектирована реляционная схема из пяти связанных сущностей: `Category` (категория), `Product` (товар), `Customer` (покупатель), `Order` (заказ) и `OrderItem` (позиция заказа). Связи реализованы через внешние ключи: один-ко-многим (`Category->Product`, `Customer->Order`) и многие-ко-многим через связующую таблицу (`Order<->Product` через `OrderItem`).
3. **Создание моделей и DbContext.** Для каждой сущности создан C#-класс с атрибутами валидации (`[Required]`, `[StringLength]`, `[Range]`, `[EmailAddress]`) и навигационными свойствами. Создан класс `StoreContext`, наследующий `DbContext`, с регистрацией `DbSet<>` для всех сущностей. В методе

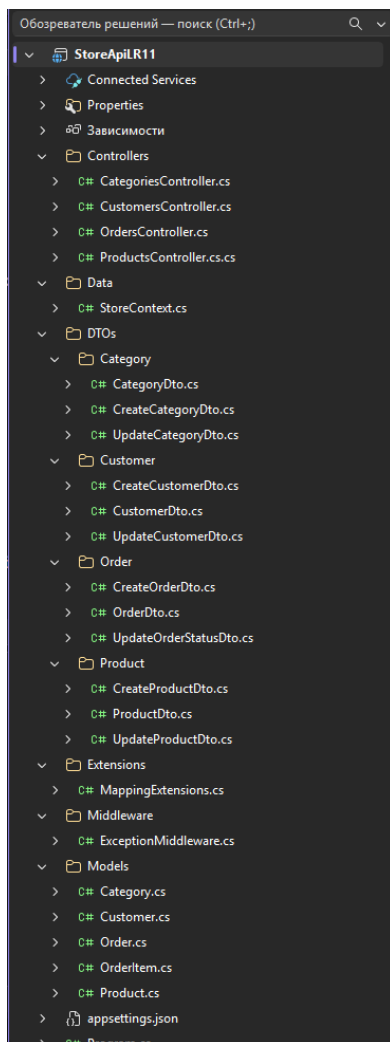
OnModelCreating настроены связи, правила каскадного удаления (OnDelete) и ограничения целостности данных. Подключение к локальному серверу MySQL сконфигурировано через appsettings.json и зарегистрировано в DI-контейнере с провайдером UseMySQL.

4. **Реализация DTO-слоя.** Для разделения внутренних моделей БД и контрактов API созданы отдельные DTO-классы: *Dto для ответов, Create*Dto и Update*Dto для запросов. В DTO добавлены атрибуты валидации, что предотвращает over-posting и упрощает обработку ошибок на уровне ModelState. Реализованы методы-расширения для маппинга сущностей <-> DTO без использования сторонних библиотек.
5. **Реализация полного CRUD для всех сущностей.** Для каждой из пяти сущностей (Categories, Products, Customers, Orders, OrderItems) реализован отдельный контроллер с полным набором HTTP-методов: GET (список/по ID), POST (создание), PUT (обновление), DELETE (удаление). Все операции работают асинхронно, возвращают корректные статус-коды (200, 201, 204, 400, 404, 500) и используют внедрение StoreContext через конструктор.
6. **Основная бизнес-логика: создание заказа.** Метод CreateOrder в OrdersController является ключевой фишкой проекта. Его реализация включает:
 - **Транзакционное выполнение:** Используется паттерн DbContext.Database.CreateExecutionStrategy().ExecuteAsync() для совместимости ручной транзакции с retry-стратегией MySQL (EnableRetryOnFailure).
 - **Атомарность:** Проверка клиента, валидация остатков на складе, списание товара, фиксация цены на момент заказа и сохранение заголовка заказа с позициями происходят в одной транзакции. При любой ошибке выполняется Rollback, данные не попадают в БД.

- **Защита от гонок (Race Conditions):** Цена товара фиксируется в `OrderItem.Price` на момент создания заказа, а остаток `Stock` уменьшается синхронно с `SaveChangesAsync()`.

7. Дополнительные архитектурные улучшения. Реализованы:

- **Soft Delete:** Добавлены поля `IsDeleted` и `DeletedAt`. В `StoreContext` настроены глобальные query-фильтры, автоматически исключающие помеченные записи из всех запросов.
- **Пагинация, фильтрация и сортировка:** `GET /api/Products` поддерживает query-параметры `?page=1&pageSize=20&sortBy=price&sortOrder=desc&categoryId=1`.
- **Логирование:** Внедрён `ILogger<T>` для отслеживания этапов создания заказа, валидации остатков и обработки исключений.



Изображение 1 Структура проекта

2. Запуск приложения и тестирование

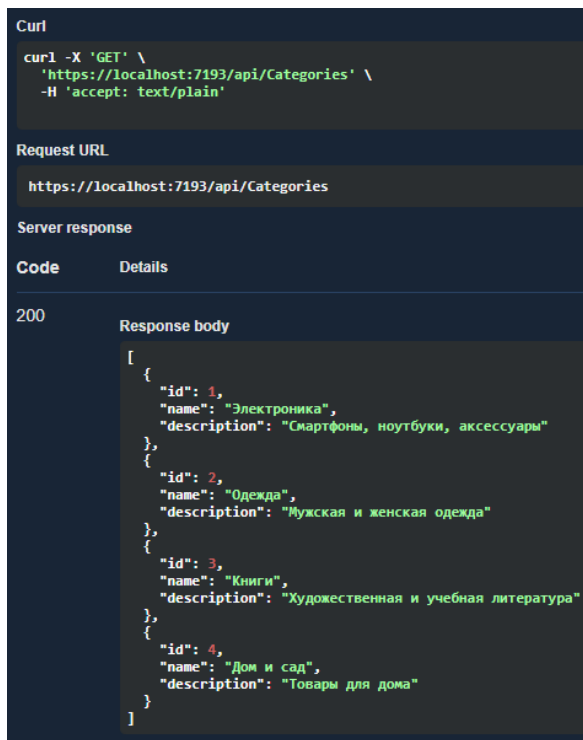
Для тестирования функционала API использовался **Swagger UI**.

Таблица 1 Реализованные API-эндпоинты (полный CRUD для всех сущностей)

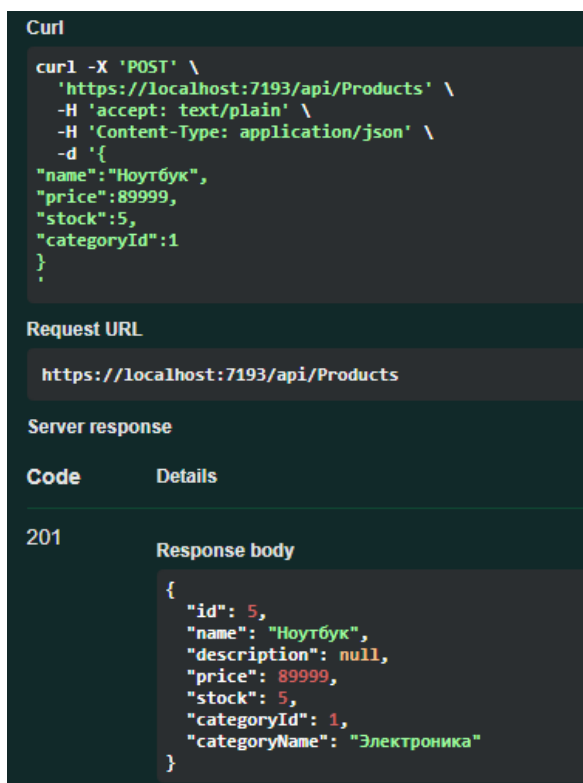
Сущность	GET	POST	PUT	DELETE	Дополнительно
Categories	/api/Categories /api/Categories/{id}	/api/Categories	/api/Categories/{id}	/api/Categories/{id}	Soft Delete
Products	/api/Products /api/Products/{id}	/api/Products	/api/Products/{id}	/api/Products/{id}	Фильтрация, пагинация, сортировка
Customers	/api/Customers /api/Customers/{id}	/api/Customers	/api/Customers/{id}	/api/Customers/{id}	Email-валидация
Orders	/api/Orders /api/Orders/{id}	/api/Orders	/api/Orders/{id}/status	/api/Orders/{id}	Транзакция, списание остатков

Таблица 2 Результаты тестирования ключевых сценариев

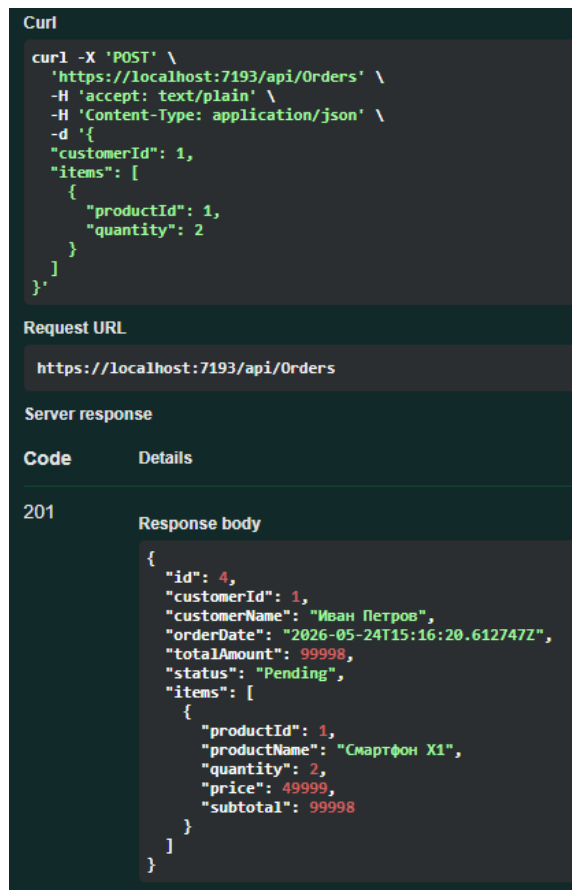
№	URL	HTTP	Параметры	результат
1	/api/Categories	GET	-	200 OK, массив категорий
2	/api/Products	POST	{"name":"Ноутбук","price":89999,"stock":5,"categoryId":1}	201 Created, объект с сгенерированным Id
3	/api/Orders	POST	{"customerId":1,"items":[{"productId":1,"quantity":2}]}	201 Created, товар списан, заказ сохранён
4	/api/Orders	POST	{"customerId":1,"items":[{"productId":1,"quantity":999}]}	400 Bad Request, ошибка "Not enough stock"
5	/api/Orders/1/status	PUT	{"status":"Processing"}	204 No Content
6	/api/Products/999	GET	несуществующий ID	404 Not Found



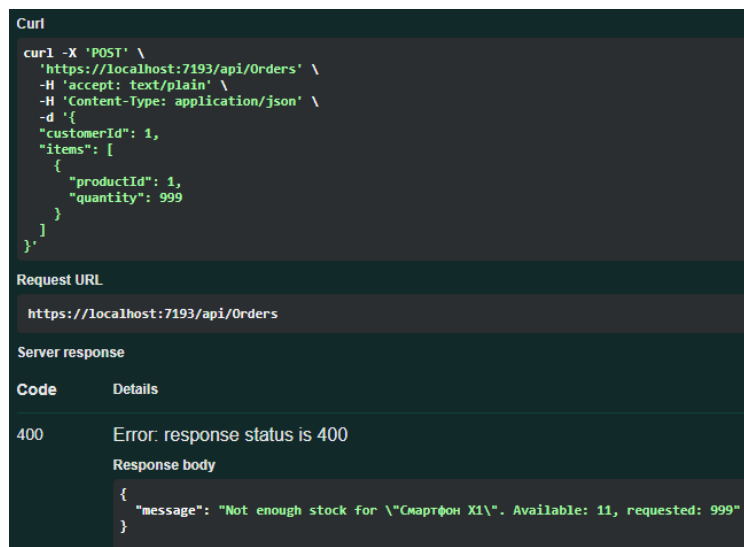
Изображение 2 GET api/Categories



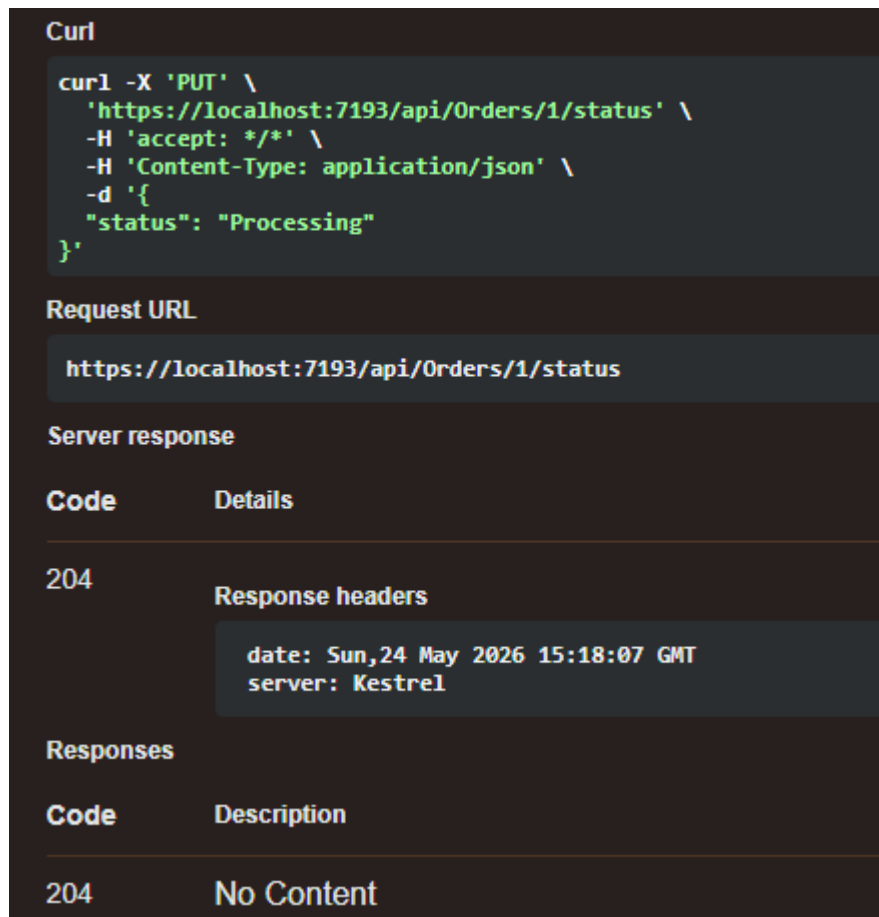
Изображение 3 POST /api/Products



Изображение 4 POST /api/Orders



Изображение 5 POST /api/Orders



Изображение 6 PUT /api/Orders/1/status



Изображение 7 GET /api/Products/999

3. Листинг кода

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Сервисы
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Подключение к MySQL через EF Core
builder.Services.AddDbContext<StoreContext>(options =>
    options.UseMySQL(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        ServerVersion.AutoDetect(builder.Configuration.GetConnectionString("DefaultConnection"))
    );

var app = builder.Build();

// Middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseMiddleware<ExceptionMiddleware>();
app.MapControllers();
app.Run();
```

Контекст базы данных и настройка связей Data/StoreContext.cs

```
public class StoreContext : DbContext
{
    public StoreContext(DbContextOptions<StoreContext> options) : base(options) { }

    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Customer> Customers => Set<Customer>();
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Order <--> Customer (Cascade)
        modelBuilder.Entity<Order>()
            .HasOne(o => o.Customer).WithMany(c => c.Orders)
            .HasForeignKey(o => o.CustomerId).OnDelete(DeleteBehavior.Cascade);

        // OrderItem <--> Order (Cascade)
        modelBuilder.Entity<OrderItem>()
            .HasOne(oi => oi.Order).WithMany(o => o.OrderItems)
            .HasForeignKey(oi => oi.OrderId).OnDelete(DeleteBehavior.Cascade);

        // OrderItem <--> Product (Restrict: запрет удаления товара, если есть заказы)
        modelBuilder.Entity<OrderItem>()
            .HasOne(oi => oi.Product).WithMany(p => p.OrderItems)
            .HasForeignKey(oi => oi.ProductId).OnDelete(DeleteBehavior.Restrict);
    }
}
```

Транзакционная логика создания заказа Controllers/OrdersController.cs

```
[HttpPost]
public async Task<ActionResult<OrderDto>> CreateOrder(CreateOrderDto dto)
{
    // Получаем execution strategy для корректной работы с retry-политикой MySQL
    var strategy = _context.Database.CreateExecutionStrategy();
    return await strategy.ExecuteAsync(async () =>
    {
        using var transaction = await _context.Database.BeginTransactionAsync();
        try
        {
            var order = new Order { CustomerId = dto.CustomerId, OrderDate =
DateTime.UtcNow, Status = "Pending", OrderItems = new List<OrderItem>() };
            decimal total = 0;

            foreach (var itemDto in dto.Items)
            {
                var product = await _context.Products.FindAsync(itemDto.ProductId);
                if (product.Stock < itemDto.Quantity)
                    return BadRequest(new { message = $"Недостаточно товара:
{product.Name}" });

                product.Stock -= itemDto.Quantity;
                _context.Products.Update(product);

                order.OrderItems.Add(new OrderItem { ProductId = itemDto.ProductId,
Quantity = itemDto.Quantity, Price = product.Price });
                total += itemDto.Quantity * product.Price;
            }

            order.TotalAmount = total;
            _context.Orders.Add(order);
            await _context.SaveChangesAsync(); // Атомарное сохранение всех изменений
            await transaction.CommitAsync();

            var created = await _context.Orders.Include(o => o.Customer).Include(o =>
o.OrderItems).ThenInclude(oi => oi.Product).FirstAsync(o => o.Id == order.Id);
            return CreatedAtAction(nameof(GetOrder), new { id = created.Id },
created.ToDto());
        }
        catch
        {
            await transaction.RollbackAsync(); // Откат при любой ошибке
            throw;
        }
    });
}
```

DTO-контракты **и** **маппинг** **DTOs/CreateOrderDto.cs,**
Extensions/MappingExtensions.cs

```

// DTO с валидацией
using StoreApiLR11.DTOS;
using StoreApiLR11.DTOS.Product;
using StoreApiLR11.Models;
using System.ComponentModel.DataAnnotations;

public record CreateOrderDto(
    [Range(1, int.MaxValue)] int CustomerId,
    [Required, MinLength(1)] List<CreateOrderItemDto> Items);

public record CreateOrderItemDto(
    [Range(1, int.MaxValue)] int ProductId,
    [Range(1, 1000)] int Quantity);

// Ручной маппинг сущностей в DTO
public static class MappingExtensions
{
    public static OrderDto ToDto(this Order order) => new(
        order.Id, order.CustomerId, order.Customer?.Name ?? "Unknown",
        order.OrderDate, order.TotalAmount, order.Status,
        order.OrderItems?.Select(oi => new OrderItemDto(
            oi.ProductId, oi.Product?.Name ?? "", oi.Quantity, oi.Price, oi.Quantity *
oi.Price
        )).ToList() ?? new());

    public static void UpdateFrom(this Product entity, UpdateProductDto dto)
    {
        entity.Name = dto.Name; entity.Description = dto.Description;
        entity.Price = dto.Price; entity.Stock = dto.Stock; entity.CategoryId =
dto.CategoryId;
    }
}

```